



Tutorial S2B: Object Oriented Programming (Solutions)

Section A: Discussion Questions

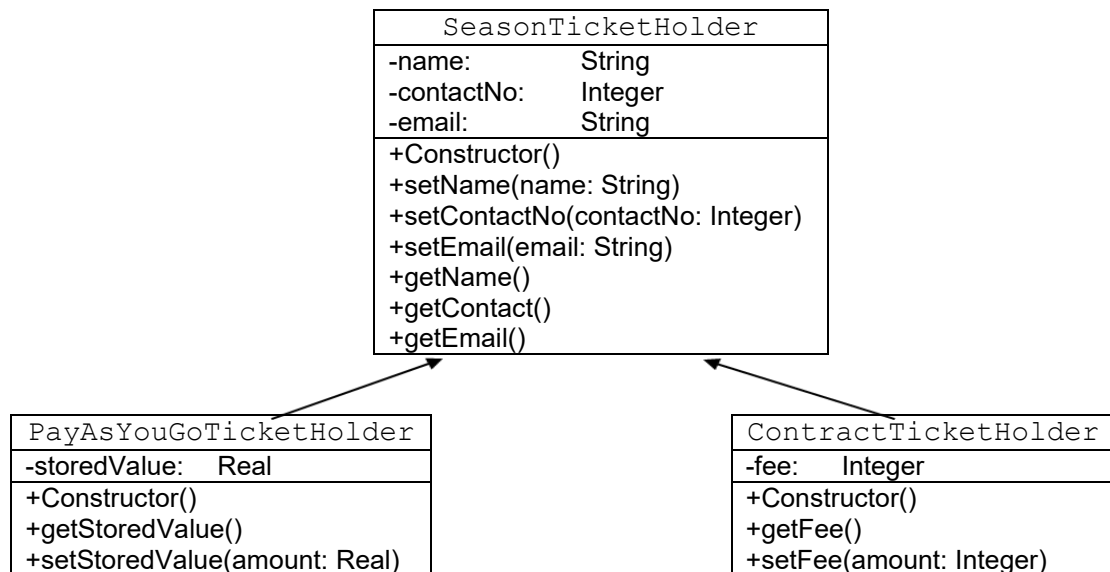
1.

```
CLASS Book
    #instance variables
    PRIVATE title
    #add missing instance variables here
    PRIVATE author
    PRIVATE isbn
    PRIVATE price
    #constructor
    PUBLIC PROCEDURE new (aTitle, aAuthor, aISBN, aPrice)
        title = aTitle
        author = aAuthor
        isbn = aISBN
        price = aPrice
    ENDPROCEDURE

    PUBLIC PROCEDURE displayDetails
        output title, author, isbn, price
    ENDPROCEDURE
ENDCLASS
```



2(a)



2(b)(i) Encapsulation is combining together attributes and methods as a single object type. All the data and methods of each object are wrapped up into a single entity so that the attributes and behaviours of one object cannot affect the way in which another object functions. For example, the `SeasonTicketHolder` class consists of private attributes (`name`, `contactNo`, `email`) and public methods (`setName`, `setContactNo`, etc). The private attributes can only be accessed through the public methods.

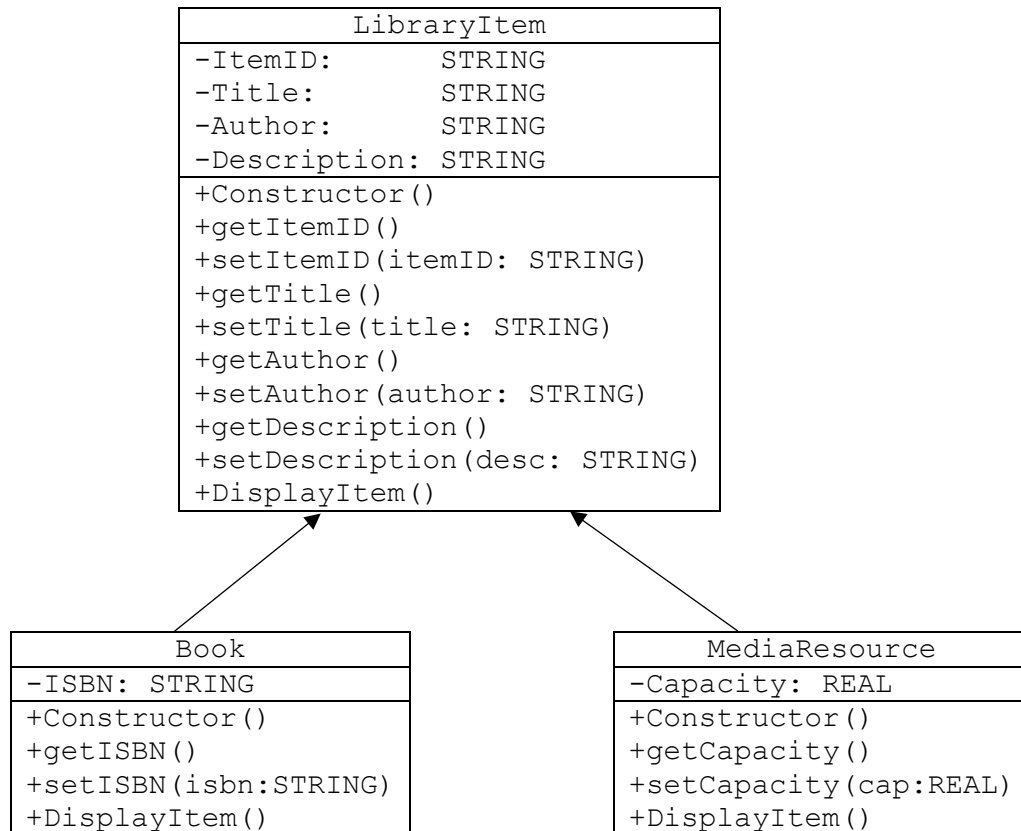
2(b)(ii) Instantiation is to create a new object from a class. The `SeasonTicketHolder` class has been defined with attributes and methods. During instantiation, the constructor new procedure initialises and sets up a new object so that it has all the attributes and methods defined in the class but the object has its own data.

2(c) As a general rule, instance variables or attributes are declared private and methods public, so that other classes may use methods belonging to another class but may not see or change their attributes. This is the principle of *information hiding*, where a class cannot directly access the attributes of another class when they are declared private.

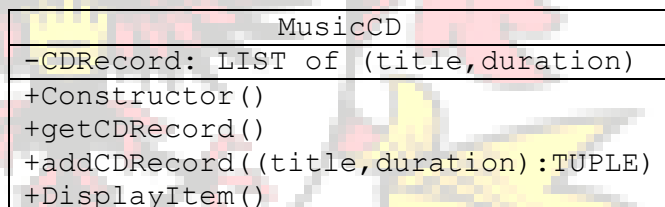
2(d) `newCustomer = new ContractTicketHolder("Johnny", 91234569, "johnnyyy@xyz.sg", 999)`



3(a) Method `DisplayItem()` demonstrates polymorphism. It exists as the same name in all 3 classes but display different attributes in each of the classes.



3(b) MusicCD is a type of media resource.
It can be categorised as a **subclass** of **MediaResource**.
Each CD should contain a **list** of songs.
With the song title and duration recorded as tuples.
A method displaying all song tracks in a given CD is to be implemented.



OR

Song class to be created with title and duration as attributes
List of song objects to be stored in MusicCD class.



3(c) OOP promotes **software reuse** as all objects in the same class shares common properties and behaviors, which are defined using attributes and methods. Each object has access to the common methods defined and this promotes software reuse.

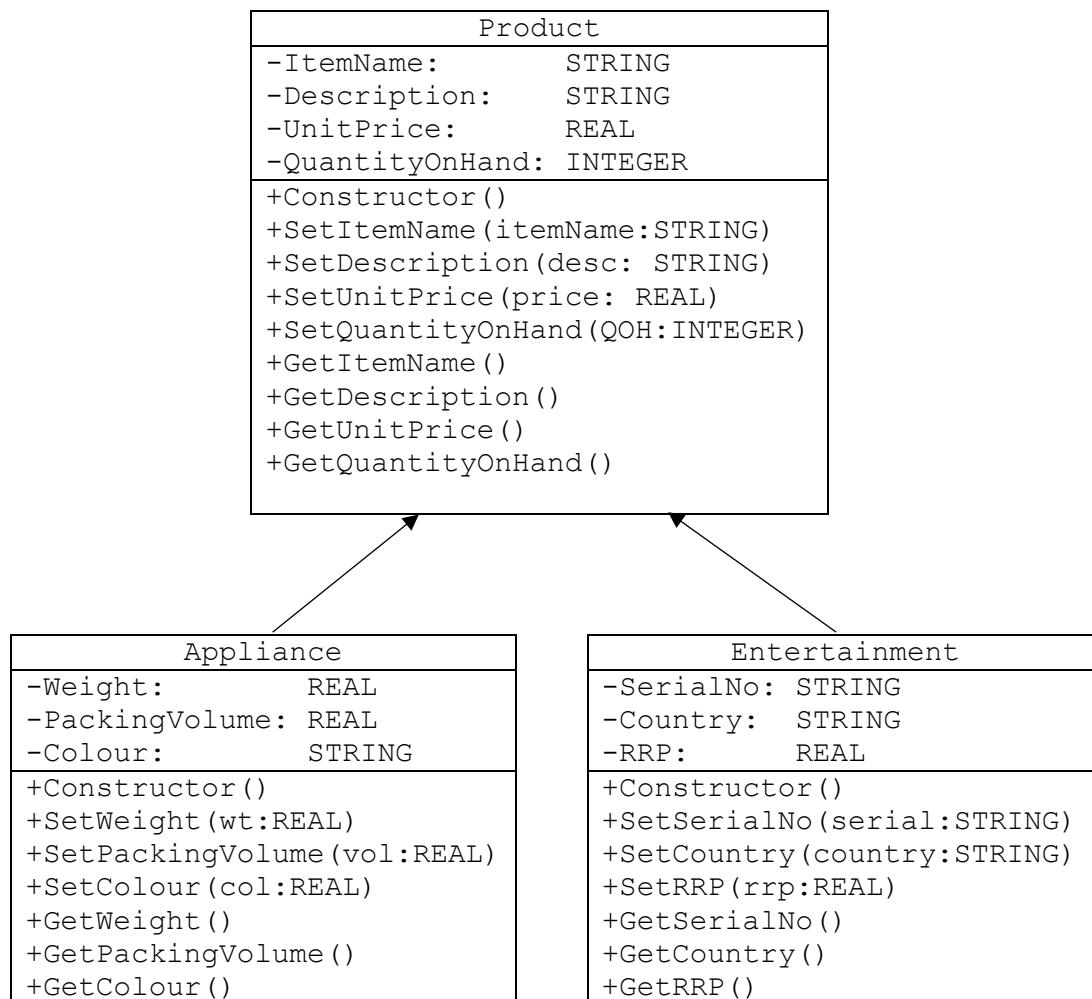
Inheritance allows common methods and data to be in the parent's class, so that the common methods do not need to be rewritten in the child classes. For example all library item will have an item ID and title data and also corresponding methods to acquire the data.

3(d) Encapsulation is combining methods and attributes into a single object type. It is used to hide the values or state of a structured data object inside a class, preventing unauthorised parties' direct access to them.

In `LibraryItem` class, attributes such as `ItemID`, `Title`, `Author` and `Description` will be stored as private attributes, while mutators and accessors such as `set_description()` and `get_libraryItemID()` is defined to modify and retrieve the data.



4(a)



4(b)(i) Encapsulation is the encasing of both private attributes (e.g. ItemName, Description and Price in class Products) and public methods (SetItemName, GetItemName ...) of a class within a container (in this case, the elements within a class definition) where the only access is through a well-defined interface (e.g. SetUnitPrice, GetUnitPrice).

Encapsulation provides the mechanism for data hiding, where the data and operations of a class can only be accessed according to the level that it was specified for, e.g. private elements cannot be accessed from outside the class while public elements can be accessed. Private data can be assessed by outside functions only through the class' public member functions. E.g. to access the ItemName of class Products, other functions have to call GetItemName to do so. In this way data hiding is achieved where ItemName cannot be accessed directly by other functions not in the class.

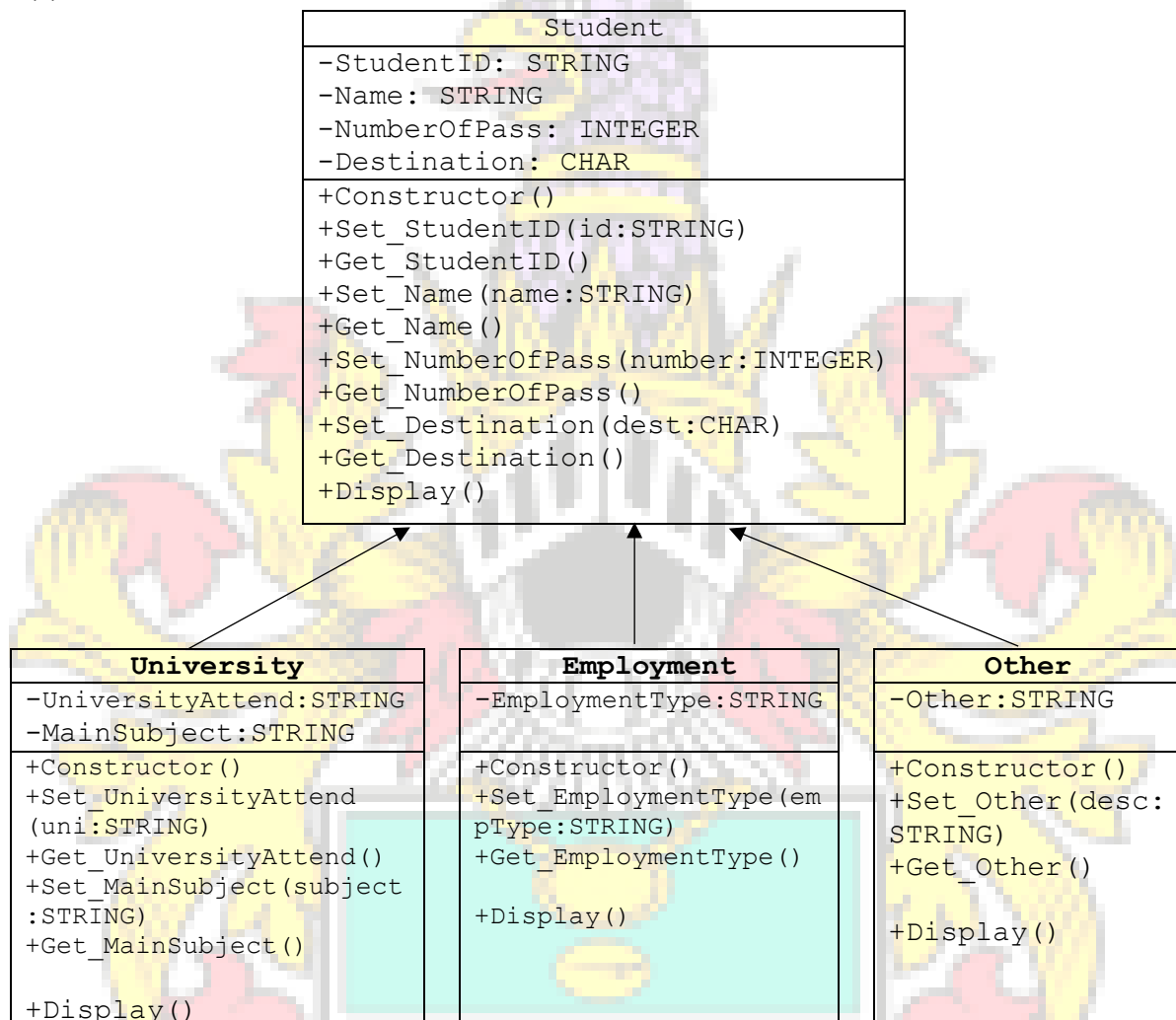


4(b)(ii) Inheritance is the ability to create new classes by inheriting the data and operations of existing classes. The shared knowledge of common traits forms the basis for inheritance relationships, usually hierarchical from general to more specific classes. For instance class Appliances inherits from Products all its public methods and private attributes and in addition it has specific attributes of Weight, PackingVolume and Colour, while class Entertainment has additional attributes serialNo, Country, RRP.

4(b)(iii) Polymorphism is the ability of a methods of the same name to behave according to its context, e.g. the Operate method for Appliance will behave differently from the Operate method of the same name in Entertainment. One may operate an oven while the other plays visual and audio of a TV for instance.



5(a)



5(b) There are various contexts in which reuse are possible:

- *Encapsulation allows us to reuse an abstract data type to create multiple objects.* For example, multiple student objects can be created, one for each student in the school.
- **Inheritance** allows classes to be reused in the definition of other classes. For example, the University, Employment and Other classes inherited the Student base class and hence, reused its attributes like StudentID and Name. Inheritance provides an effective method of code reuse because the common operations defined and implemented in a class can be accessed by all the classes and instances down the whole class hierarchy. As and when new classes inherit from existing classes, the code developed in super-classes get reused. This feature leads to tremendous saving of programming effort.



- *Polymorphism means the reuse of symbols (operators and names) to apply to different object behaviours.* For example, Display is being reused in different sub classes.

There is no need to reinvent the wheel when it comes to creating different applications. Code reusability features also help in making software systems easy to maintain. This is because by making a change in a super-class, in effect all its sub-classes are changed at once, without the tedium of identifying and implementing the same changes at a number of places.

5(c) To read data from STUDENT.DAT and output the following:

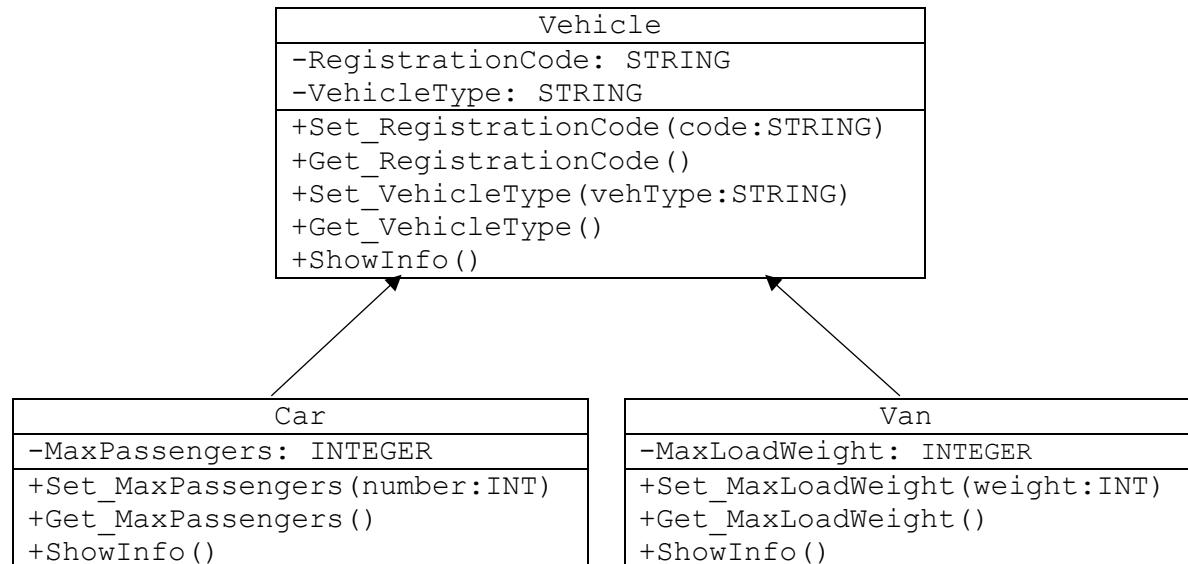
- Total number of students going to University
- Average number of passes for students going to University
- Total number of students

```
BEGIN PROCEDURE
  OPEN STUDENT.DAT for INPUT
  DECLARE TotalStudent, TotalUniversity, TotalPassesUniversity: INTEGER
  SET TotalStudent = 0
  SET TotalUniversity = 0
  SET TotalPassesUniversity = 0
  FOR each line in STUDENT.DAT
    INCREMENT TotalStudent by 1
    FOR each field in each line
      IF Destination = 'U' THEN
        INCREMENT TotalUniversity by 1
        TotalPassesUniversity = TotalPassesUniversity + NoOfPasses
      ENDIF
    ENDFOR
  ENDFOR
  CLOSE STUDENT.DAT
  OUTPUT "Total number of students going to university", TotalUniversity
  OUTPUT "Average number of passes for students going to university",
    TotalPassesUniversity / TotalUniversity
  OUTPUT "Total number of students", TotalStudent
END PROCEDURE
```




Section B (Assignment Question)

1(a)



1(b) EITHER

- minibus is nether car nor van...
- so new class minibus with parent class Vehicle
- new class has property MinibusType / NoOfSeats ...
- restricted to limited set of value (6,12,14)
- new class has methods for setting and getting new property
- possible values for VehicleType increased to include Minibus

OR

- minibus and car both carry passengers
- only difference is maximum number of passengers
- so new class PassengerVehicle with parent class Vehicle
- new class has property of MaxPassengers ...
- has methods for setting and getting MaxPassengers and ShowInfo
- possible values for VehicleType increased to include Minibus

1(c) Data (attributes) and methods to manipulate data held in same structure.

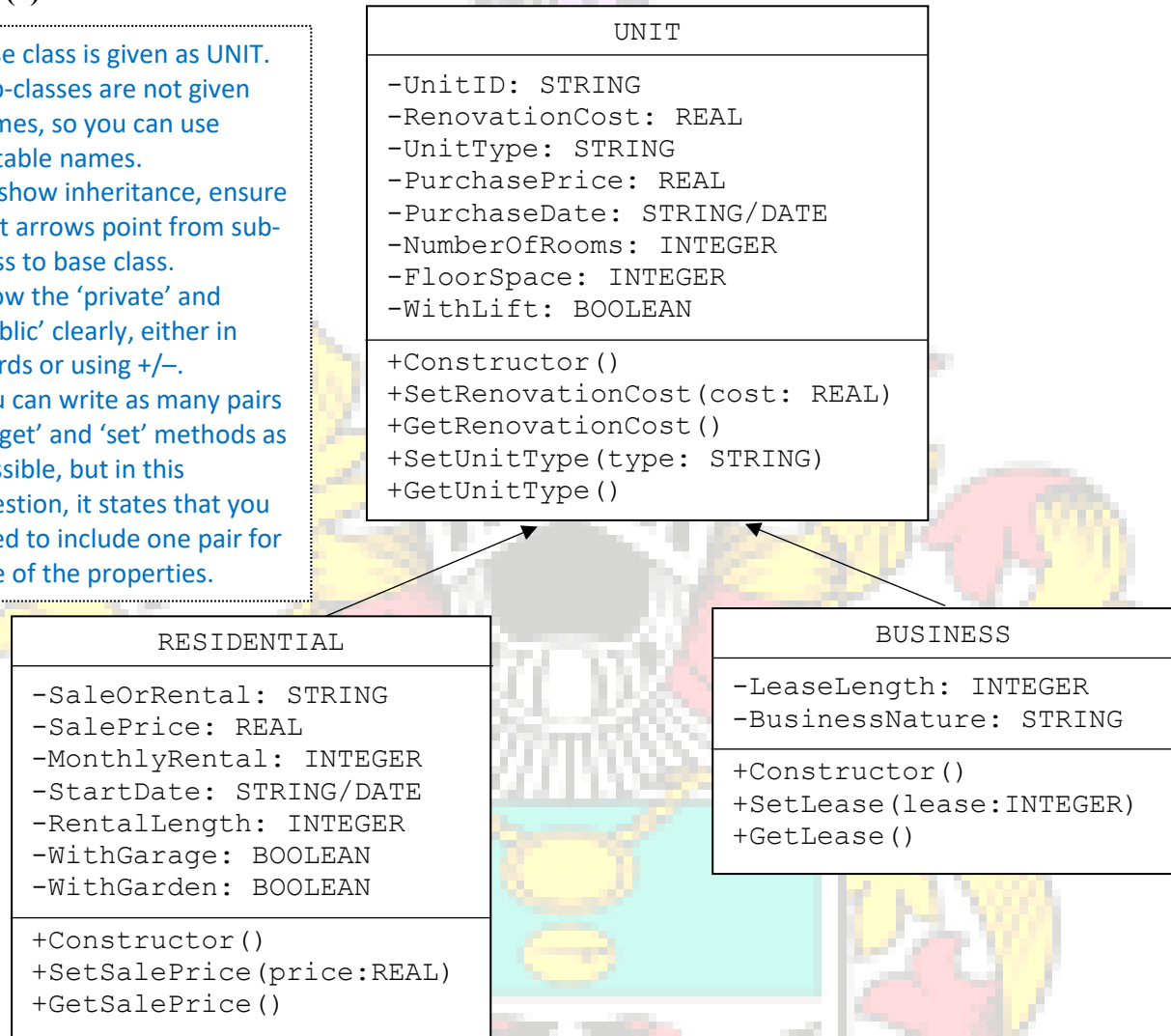
Implementation details hidden from user and data (attributes) only accessible via class interface (methods)

e.g. setting VehicleType (`Set_VehicleType`) includes check for valid value (either Car, Van or Minibus) – handling new values does not require alteration of method calls



2(a)

Base class is given as UNIT.
Sub-classes are not given names, so you can use suitable names.
To show inheritance, ensure that arrows point from sub-class to base class.
Show the 'private' and 'public' clearly, either in words or using +/-.
You can write as many pairs of 'get' and 'set' methods as possible, but in this question, it states that you need to include one pair for one of the properties.



2(b) Since a flat is a residential unit, RESIDENTIAL class can introduce properties BlockNumber to indicate the 'block of flats' which the residential unit belongs to, and a BlockPrice to indicate the price of the 'block of flats'. If the flat does not belong to any block, then a default value such as 'Not Applicable' can be applied to the BlockNumber and BlockPrice. Additional methods SetBlockNumber and GetBlockNumber to set and get the block number. Also SetBlockPrice and GetBlockPrice to set and get the block price.

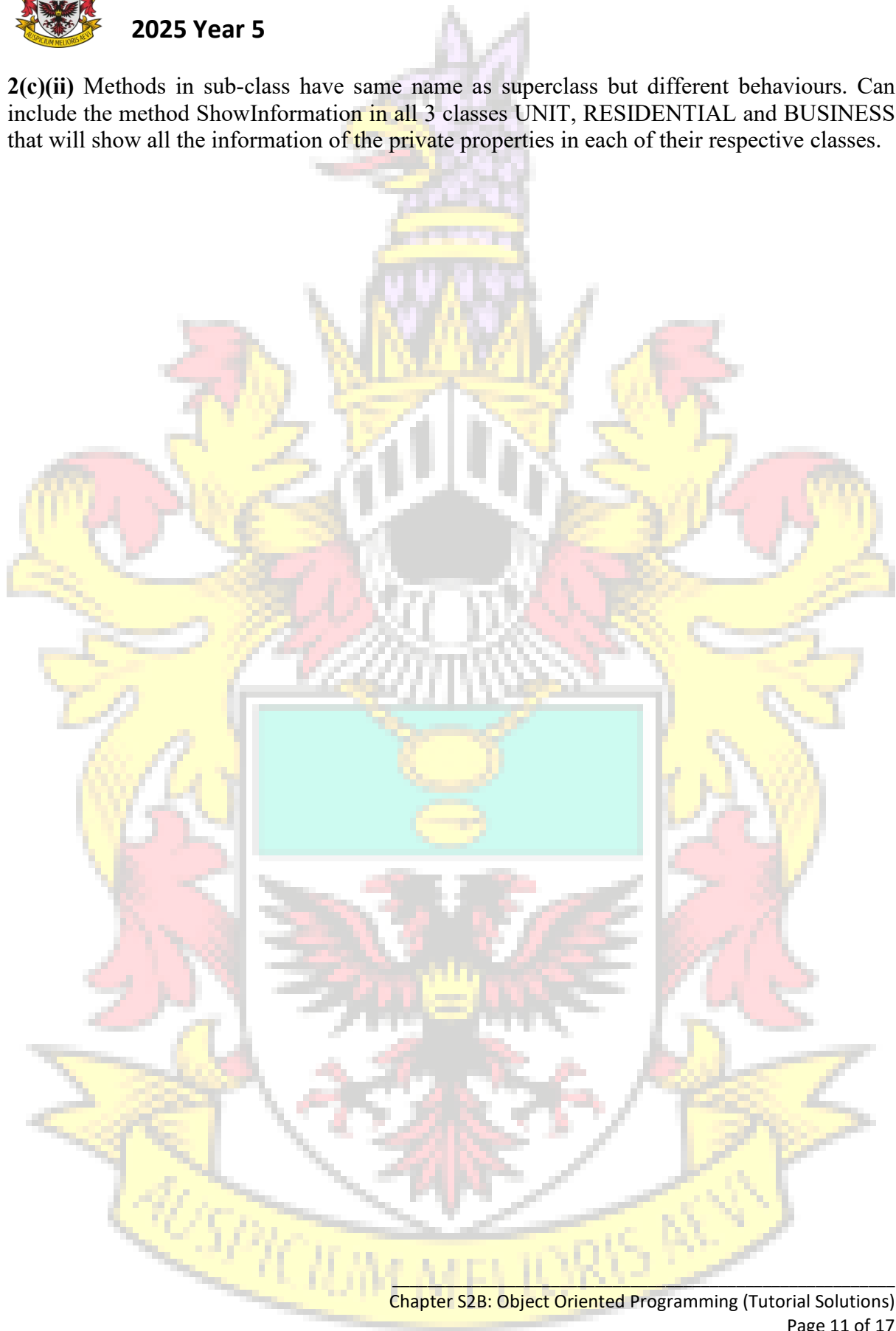
2(c)(i) Private properties are only accessed or altered by calls to public methods. For example, SalePrice is a private property in RESIDENTIAL class that can only be accessed or altered by SetSalePrice and GetSalePrice.



RAFFLES INSTITUTION H2 Computing (9569)

2025 Year 5

2(c)(ii) Methods in sub-class have same name as superclass but different behaviours. Can include the method `ShowInformation` in all 3 classes `UNIT`, `RESIDENTIAL` and `BUSINESS` that will show all the information of the private properties in each of their respective classes.





S2B: Object Oriented Programming (Programming Exercises)

1

LibraryItem	
-Title	: STRING
-Author_Artist	: STRING
-ItemID	: INTEGER
-OnLoan	: BOOLEAN
-DueDate	: DATE
+Constructor()	
+GetTitle()	
+GetAuthorArtist()	
+GetItemID	
+GetOnLoan()	
+GetDueDate()	
+Borrowing()	
+Returning()	
+PrintDetails()	

Book	
-IsRequested: BOOLEAN	
+Constructor()	
+GetIsRequested()	
+SetIsRequested()	

CD	
-Genre: STRING	
+Constructor()	
+GetGenre()	
+SetGenre()	

```
import datetime
class LibraryItem:
    def __init__(self, title, author_artist, item): #constructor
        self.__Title = title
        self.__Author_Artist = author_artist
        self.__ItemID = item
        self.__OnLoan = False
        self.__DueDate = datetime.date.today()

    def GetTitle(self):
        return self.__Title
    def GetAuthorArtist(self):
        return self.__Author_Artist
    def GetItemID(self):
        return self.__ItemID
    def GetOnLoan(self):
        return self.__OnLoan
    def GetDueDate(self):
        return self.__DueDate
    def Borrowing(self):
        self.__OnLoan = True
        self.__DueDate = self.__DueDate + datetime.timedelta(weeks=3)
    def Returning(self):
        self.__OnLoan = False
```



```
def PrintDetails(self):
    print("Title =",self.GetTitle())
    print("Author/Artist =",self.GetAuthorArtist())
    print("ItemID =",self.GetItemID())
    print("On Loan =",self.GetOnLoan())
    print("Due Date =",self.GetDueDate())

class Book(LibraryItem):
    def __init__(self, title, author_artist, item): #constructor
        super().__init__(title, author_artist, item)
        self.__IsRequested = False
        self.__RequestedBy = 0
    def GetIsRequested(self):
        return self.__IsRequested
    def SetIsRequested(self):
        self.__IsRequested = True

class CD(LibraryItem):
    def __init__(self, title, author_artist, item): #constructor
        LibraryItem.__init__(self, title, author_artist, item)
        self.__Genre = ""
    def GetGenre(self):
        return self.__Genre
    def SetGenre(self, genre):
        self.__Genre = genre
```

2 (a)

```
class Attendance:
    def __init__(self,date,status):
        self.__Date = date
        self.__Status = status
    def GetStatus(self):
        return self.__Status

class Student:
    def __init__(self,name,formClass):
        self.__Name = name
        self.__FormClass = formClass
        self.__AttendanceRecord = [] #empty list at initial

    def AddAttendance(self, record):
        self.__AttendanceRecord.append(record) #record is an Attendance object

    def CountAbsent(self): #count number of days ABSENT
        count = 0
        for eachRec in self.__AttendanceRecord: #iterate thru list of AttendanceRecord
            if eachRec.GetStatus() == "ABSENT": #check for ABSENT status
                count += 1
        return count
```



```
def CountLate(self): #count number of days ABSENT
    count = 0
    for eachRec in self.__AttendanceRecord: #iterate thru list of AttendanceRecord
        if eachRec.GetStatus() == "LATE": #check for LATE status
            count += 1
    return count

def Display(self):
    print("{} , {} , was absent on {} day(s) and late on {} day(s)." .\
        format(self.__Name, self.__FormClass, self.CountAbsent(), self.CountLate()))

## Test driver code
a1=Attendance("01/08/2021", "PRESENT")
a2=Attendance("02/08/2021", "LATE")
a3=Attendance("03/08/2021", "ABSENT")
student = Student("Johnny Lim", "23S99")
student.AddAttendance(a1)
student.AddAttendance(a2)
student.AddAttendance(a3)
absent = student.CountAbsent()
late = student.CountLate()
student.Display()
```

2 (b)

```
def AddRecords():
    inFile = open("ATT.txt", 'r')
    student_data = inFile.readline().strip()
    name, formclass = student_data.split(',')
    student = Student(name, formclass)
    for line in inFile:
        line = line.strip()
        date, status = line.split(',')
        attendance = Attendance(date, status)
        student.AddAttendance(attendance)

    return student

#=====
a1=Attendance("01/08/2021", "PRESENT")
a2=Attendance("02/08/2021", "LATE")
a3=Attendance("03/08/2021", "ABSENT")
student = Student("Johnny Lim", "23S99")
student.AddAttendance(a1)
student.AddAttendance(a2)
student.AddAttendance(a3)
absent = student.CountAbsent()
late = student.CountLate()
student.Display()
```



3

```
import random

#Task (a)
class Food():
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def get_name(self):
        return self.name

    def get_value(self):
        return self.value

#Task (b)
class Person():
    def __init__(self, name):
        self.name = name
        self.health = 100
        self.strength = 100

    def get_name(self):
        return self.name

    def get_health(self):
        return self.health

    def get_strength(self):
        return self.strength

    def eat(self, food):
        to_add = food.get_value()
        self.strength += to_add
        print("Your strength is now "+str(self.get_strength()))

    def attack(self, opponent):
        value = random.randint(1,10)
        if self.strength < value:
            print("You don't have enough strength to attack. Try again later.")
        else:
            opponent.health -= value
            opponenthealth = opponent.get_health()
            if opponenthealth < 0:
                print(opponent.get_name()+" has been defeated.")
            )
        else:
            print(opponent.get_name()+"'s strength is now "+str(opponenthealth)+".")
```



```
        self.strength -= value #Player's strength decrease

#Task (c)
class Healer(Person):
    def __init__(self, name): #Healer's constructor
        super().__init__(name) #call constructor of Person

    def heal(self, patient):
        value = random.randint(1,10)
        if self.get_strength() < value:
            print("You don't have enough strength to heal. Try
again later.")
        else:
            patienthealth = patient.get_health()
            if patienthealth < 100-value:
                patient.health += value
            else:
                patient.health = 100
            print(patient.get_name()+"'s health is now "+str(pa
tienthealth))

#Task (d)
class Warrior(Person):
    def __init__(self, name): #Warrior's constructor
        super().__init__(name) #call constructor of Person

    def attack(self, opponent):
        value = random.randint(1,10)
        if self.get_strength() < value:
            print("You don't have enough strength to attack. Tr
y again later.")
        else:
            opponent.health -= 2*value
            opponenthealth = opponent.get_health()
            if opponenthealth < 0:
                print(opponent.get_name()+" has been defeated."
)
            else:
                print(opponent.get_name()+"'s strength is now "
+str(opponenthealth)+".")
                self.strength -= value

#Task (e) Testing code
Cheese = Food("Cheese", 10)
Sam = Warrior("Sam")
Alex = Healer("Alex")
Jan = Person("Jan")
```




RAFFLES INSTITUTION

H2 Computing (9569)

2025 Year 5

```
Jan.attack(Sam)  
Sam.attack(Jan)  
Alex.heal(Jan)  
Sam.eat(Cheese)
```

